NAME:- Farooqui Shahzeb Ahmed

ROLL.NO:- SYA-29

ASSINGMENT:-09

AIM:- Building a Simple Data Science Pipeline: Integrate all learned techniques in previous labs into a complete workflow

THEORY

A simple data science pipeline represents a structured workflow that integrates all essential stages of data analysis into a cohesive process, transforming raw data into meaningful insights and actionable outcomes. It typically begins with data collection from various sources, followed by data cleaning and preprocessing, where missing values, inconsistencies, and outliers are handled to ensure data quality. Next comes exploratory data analysis (EDA), which involves summarizing and visualizing data to understand patterns, trends, and relationships. Feature selection and engineering are then applied to improve model performance by identifying the most relevant variables. After this, appropriate statistical or machine learning models—such as correlation analysis, regression techniques, or classification algorithms—are selected and trained on the prepared data. Model evaluation is carried out using suitable metrics to assess accuracy and reliability, ensuring the model generalizes well to new data. Finally, the pipeline may include deployment and interpretation stages, where results are communicated effectively through reports or dashboards, enabling decision-making. In conclusion, building a simple data science pipeline ensures a systematic, repeatable, and efficient approach to problem-solving by integrating all learned techniques into a unified framework; it enhances consistency, reduces errors, and improves the overall quality of analysis, making it an essential practice for any data-driven task in real-world applications.

Q1 **What is a Data Science Pipeline? Explain its Key Stages with Examples.**

ANS:- A Data Science Pipeline is a structured, end-to-end sequence of automated steps that takes raw, unstructured data and transforms it into meaningful insights or a deployable machine learning model. It standardises the flow of data through several well-defined stages, ensuring that the entire process is reproducible, scalable, and maintainable. Just like an industrial assembly line, each stage receives input from the previous one, processes it, and passes refined output to the next — making the overall workflow systematic rather than ad hoc. The pipeline begins with Data Collection, where raw data is gathered from databases, APIs, CSV files, or web scraping. It then passes through Data Cleaning, where missing values, duplicates, and inconsistencies are handled. Exploratory Data Analysis (EDA) follows, using statistics and visualisations to understand the data. Feature Engineering transforms raw features into more informative representations. A Model is then trained, evaluated on a test set, and finally deployed into a production environment as an API or batch service. For example, an e-commerce company might collect transaction logs from MySQL, clean missing product prices, explore purchase patterns with EDA, engineer features like 'days_since_last_purchase', train a churn prediction model, evaluate it with F1-score, and deploy it as a Flask REST API that returns churn probability in real time.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.pipeline import Pipeline

from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
print('Shape:', df.shape)


print('Missing values:\n', df.isnull().sum())
df.dropna(inplace=True)
df.drop_duplicates(inplace=True)

print(df.describe())
print('Target distribution:\n', df['target'].value_counts())

X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=100, random_state=42))
])
pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
75%           0.105300          0.130400        0.130700             0.074000
max           0.163400          0.345400        0.426800             0.201200

       mean symmetry  mean fractal dimension  ...  worst texture  \
count     569.000000              569.000000  ...     569.000000
mean        0.181162                0.062798  ...      25.677223
std         0.027414                0.007060  ...       6.146258
min         0.106000                0.049960  ...      12.020000
25%         0.161900                0.057700  ...      21.080000
50%         0.179200                0.061540  ...      25.410000
75%         0.195700                0.066120  ...      29.720000
max         0.304000                0.097440  ...      49.540000

       worst perimeter   worst area  worst smoothness  worst compactness  \
count       569.000000   569.000000        569.000000         569.000000
mean        107.261213   880.583128          0.132369           0.254265
std          33.602542   569.356993          0.022832           0.157336
min          50.410000   185.200000          0.071170           0.027290
25%          84.110000   515.300000          0.116600           0.147200
50%          97.660000   686.500000          0.131300           0.211900
75%         125.400000  1084.000000          0.146000           0.339100
max         251.200000  4254.000000          0.222600           1.058000

       worst concavity  worst concave points  worst symmetry  \
count       569.000000            569.000000      569.000000
mean          0.272188              0.114606        0.290076
std           0.208624              0.065732        0.061867
min           0.000000              0.000000        0.156500
25%           0.114500              0.064930        0.250400
50%           0.226700              0.099930        0.282200
75%           0.382900              0.161400        0.317900
max           1.252000              0.291000        0.663800

       worst fractal dimension      target
count               569.000000  569.000000
mean                  0.083946    0.627417
std                   0.018061    0.483918
min                   0.055040    0.000000
25%                   0.071460    0.000000
50%                   0.080040    1.000000
75%                   0.092080    1.000000
max                   0.207500    1.000000

[8 rows x 31 columns]
Target distribution:
 target
1    357
0    212
Name: count, dtype: int64
              precision    recall  f1-score   support

           0       0.98      0.93      0.95        43
           1       0.96      0.99      0.97        71

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114
```

Q2 **What is Exploratory Data Analysis (EDA)? How do univariate and bivariate analyses help in understanding the dataset?**

ANS:- Exploratory Data Analysis (EDA) is an analytical approach, pioneered by statistician John Tukey in 1977, that involves examining datasets using visual methods and descriptive statistics before applying any formal modelling technique. Rather than jumping straight to building a model, EDA encourages the analyst to let the data reveal its own patterns, anomalies, and relationships. EDA helps understand the structure and size of the dataset, detect missing values and data quality issues, discover the distribution and central tendency of each variable, identify relationships and correlations between features, and generate hypotheses for further investigation. Without EDA, a data scientist risks building models on flawed data or selecting inappropriate algorithms.

Univariate Analysis

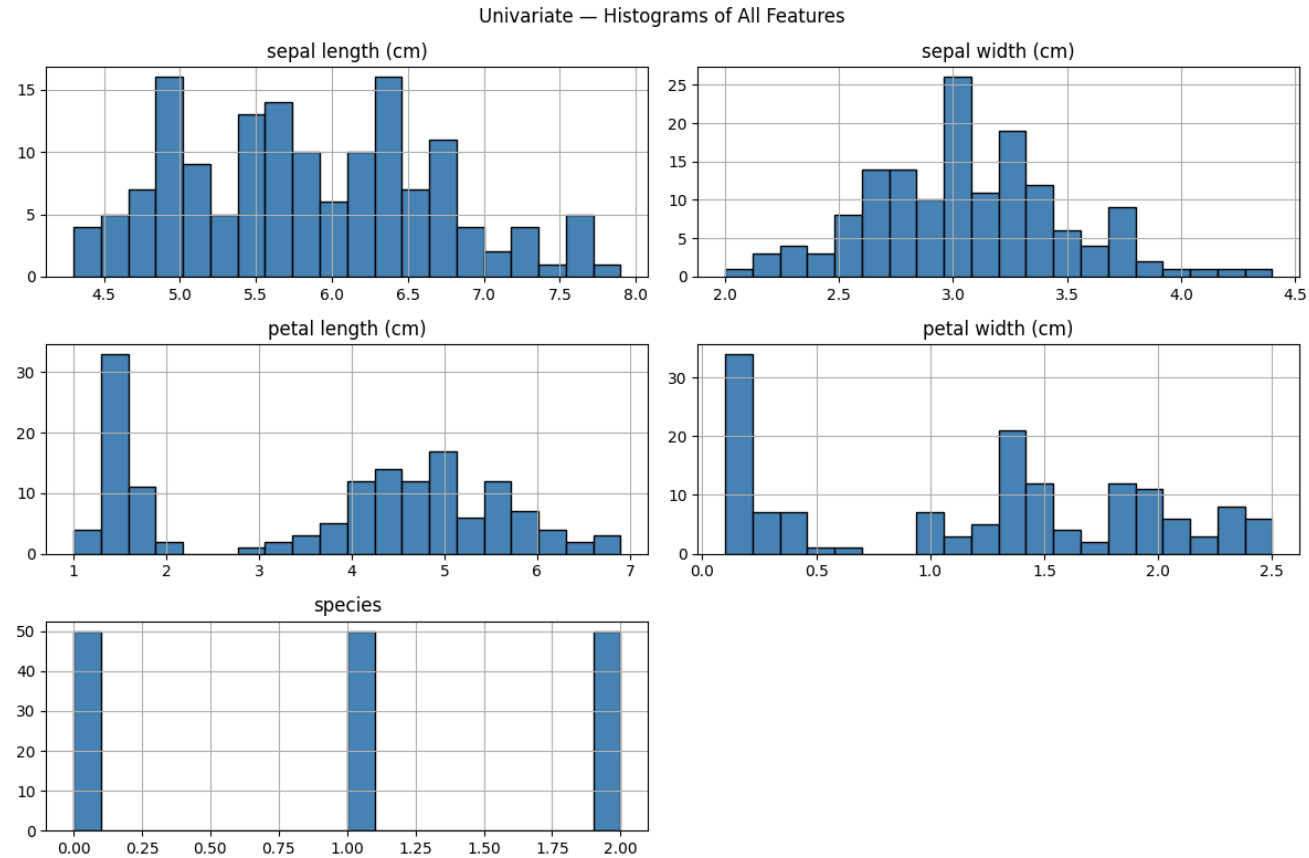
Univariate analysis examines each variable in the dataset individually and in isolation. Its purpose is to understand the distribution, central tendency (mean, median, mode), spread (variance, standard deviation, IQR), and shape (skewness, kurtosis) of a single feature at a time. For numerical variables, histograms reveal the frequency distribution and shape; box plots display the median, interquartile range, and outliers; and descriptive statistics quantify the feature numerically. For categorical variables, bar charts and value counts show how frequently each category appears, revealing class imbalance or dominant categories.

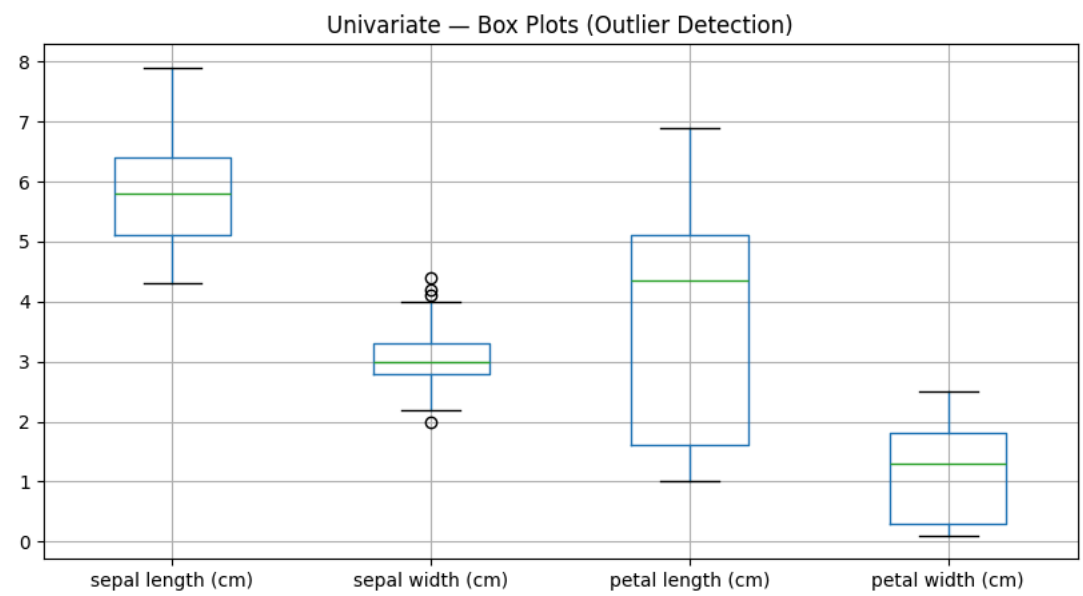
Bivariate Analysis

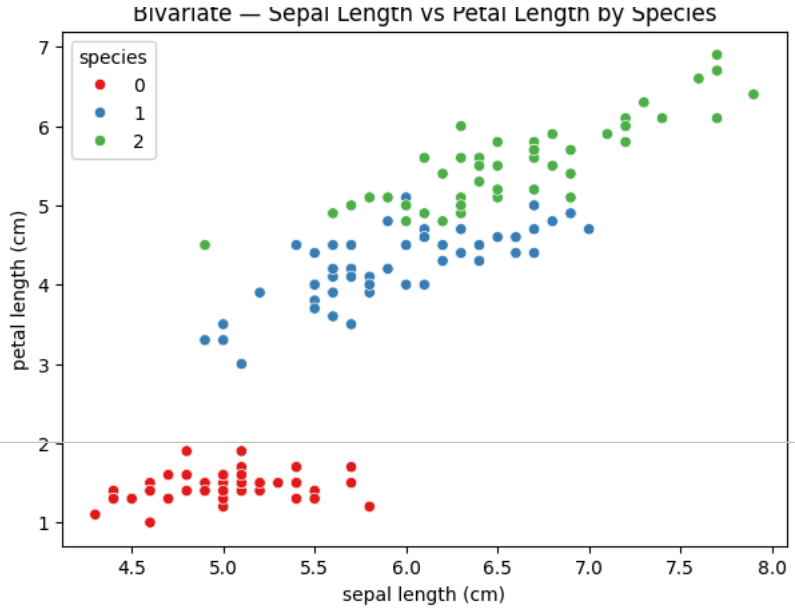
Bivariate analysis examines the relationship between two variables simultaneously. When both variables are numerical, scatter plots and correlation coefficients reveal whether a linear or non-linear relationship exists. A correlation heatmap extends this to all pairs at once, helping detect multicollinearity. When one variable is categorical and the other numerical, grouped box plots compare distributions across categories. When both are categorical, contingency tables and the Chi-Square test reveal statistical dependence. Together, univariate and bivariate analyses form the backbone of EDA — the former builds understanding of individual features, and the latter reveals interactions and dependencies essential for feature selection and model design.

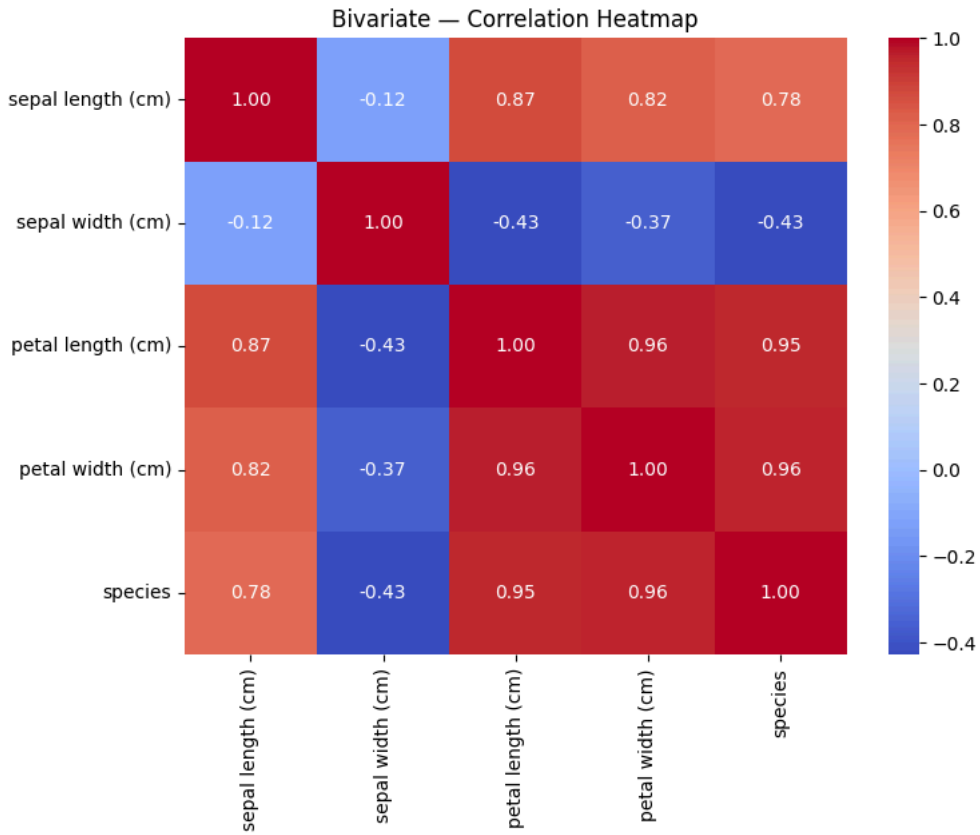
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

print('=== Descriptive Statistics ===')
print(df.describe())

df.hist(bins=20, figsize=(12, 8), color='steelblue', edgecolor='black')
plt.suptitle('Univariate — Histograms of All Features')
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))
df.drop('species', axis=1).boxplot()
plt.title('Univariate — Box Plots (Outlier Detection)')
plt.show()

plt.figure(figsize=(7, 5))
sns.scatterplot(data=df, x='sepal length (cm)',
                y='petal length (cm)', hue='species', palette='Set1')
plt.title('Bivariate — Sepal Length vs Petal Length by Species')
plt.show()

plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Bivariate — Correlation Heatmap')
plt.show()

plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x='species', y='petal length (cm)', palette='pastel')
plt.title('Bivariate — Petal Length Distribution by Species')
plt.show()
```

```
=== Descriptive Statistics ===
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)     species
count        150.000000  150.000000
mean           1.199333    1.000000
std            0.762238    0.819232
min            0.100000    0.000000
25%            0.300000    0.000000
50%            1.300000    1.000000
75%            1.800000    2.000000
max            2.500000    2.000000
```

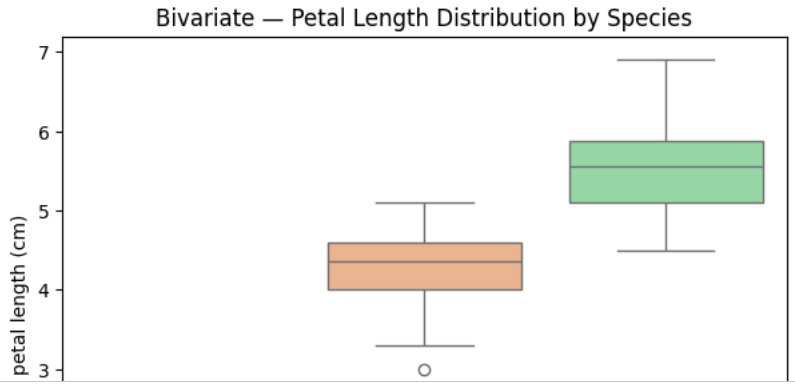






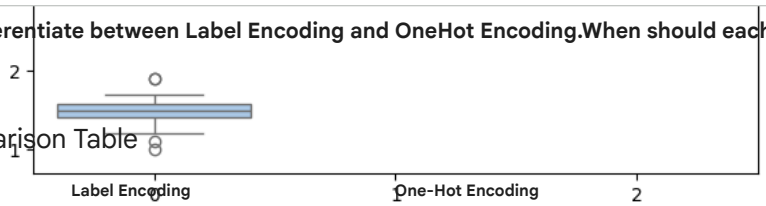



```
/tmp/ipykernel_7638/2101422456.py:36: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.boxplot(data=df, x='species', y='petal length (cm)', palette='pastel')
```



Q3 **Differentiate between Label Encoding and OneHot Encoding.When should each be used?**

ANS:-



Comparison Table

Feature	Label Encoding	One-Hot Encoding
Mechanism	Assigns a unique integer to each category.	Creates new binary columns for each category.
Output	Single numerical column.	Multiple binary (0 or 1) columns.
Ordinality	Assumes/introduces ordinal relationship.	Does not assume/introduce ordinal relationship.
Use Case	Ordinal data, high cardinality features.	Nominal data (no order), low-to-medium cardinality.
Dimensionality	No increase in feature count.	Increases feature count (can lead to sparse data).
Risk	Misleading models with artificial order.	Curse of dimensionality, multicollinearity.
Example	`Red: 0, Green: 1, Blue: 2`	`Color_Red: 1 0 0`, `Color_Green: 0 1 0`, `Color_Blue: 0 0 1`

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({
    'City':            ['Mumbai', 'Delhi', 'Pune', 'Mumbai', 'Delhi'],
    'Education_Level': ['Graduate', 'Primary', 'Post-Graduate',
                        'Secondary', 'Graduate'],
    'Salary':          [70000, 30000, 90000, 65000, 35000]
})

print('Original DataFrame:')
print(df)

order_map = {'Primary': 0, 'Secondary': 1,
             'Graduate': 2, 'Post-Graduate': 3}
df['Education_Encoded'] = df['Education_Level'].map(order_map)

print('\nAfter Label Encoding (Education_Level):')
print(df[['Education_Level', 'Education_Encoded']])

le = LabelEncoder()
df['City_LabelEncoded'] = le.fit_transform(df['City'])
print('\nLabel Encoder classes:', le.classes_)

df_ohe = pd.get_dummies(df, columns=['City'], drop_first=False)

print('\nAfter One-Hot Encoding (City):')
print(df_ohe[['City_Delhi', 'City_Mumbai', 'City_Pune',
               'Education_Encoded', 'Salary']])

df_ohe_nontrap = pd.get_dummies(df, columns=['City'], drop_first=True)
print('\nOne-Hot Encoding with drop_first=True:')
print(df_ohe_nontrap)
```

Q4 **Why is feature scaling required? Compare normalization and standardization.**

ANS:- In real-world datasets, different features are measured in entirely different units and across vastly different ranges. A dataset might simultaneously contain Age (18–80), Annual Salary (₹1,00,000–₹50,00,000), and Height (150–200 cm). When such unscaled features are fed into certain machine learning algorithms, the algorithm's computations are dominated by the feature with the largest numerical range, effectively rendering smaller-scale features irrelevant. Feature scaling is the process of bringing all numerical features onto a comparable scale so that each feature contributes proportionately to the model's learning process. Scaling is critical for algorithms that rely on distance calculations or gradient descent optimisation — including KNN, SVM, Linear Regression, Logistic Regression, and PCA. Tree-based algorithms like Decision Trees and Random Forests are the notable exceptions, as they split on individual feature thresholds and are completely scale-invariant.

Normalization (Min-Max Scaling):-

Normalization rescales all values to a fixed range [0, 1] using the formula X_norm = (X – X_min) / (X_max – X_min). The minimum value maps to 0, the maximum to 1, and all others are scaled proportionally in between. It preserves the shape of the original distribution and is ideal for algorithms requiring bounded inputs such as neural networks and image pixel normalisation. However, it is highly sensitive to outliers — a single extreme value compresses all other values into a tiny portion of the [0, 1] range.

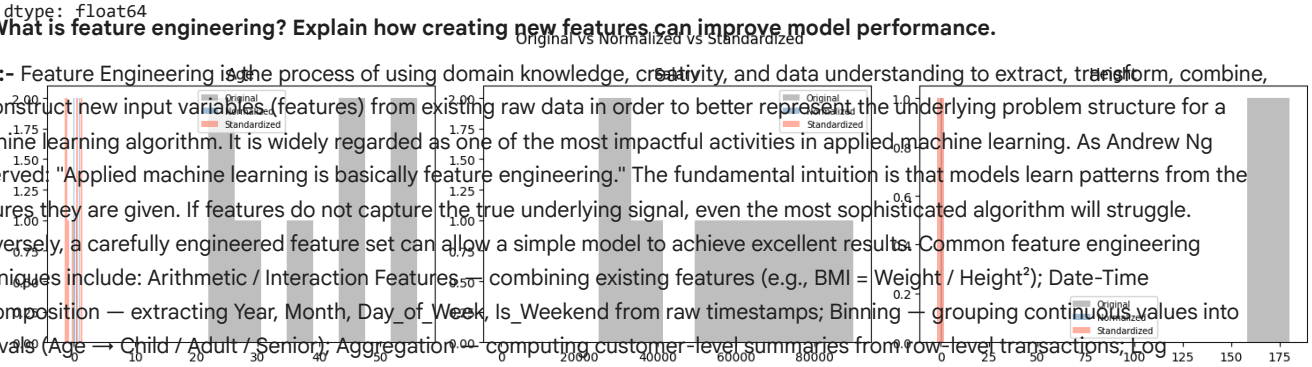
Standardization (Z-Score Scaling):-

Standardization transforms each feature to have mean = 0 and standard deviation = 1 using the formula X_std = (X − μ) / σ. Unlike normalization, the output has no fixed range and can be negative. It is more robust to outliers since an extreme value simply produces a large z-score without distorting the rest of the data. Standardization is preferred for SVM, PCA, and linear models, and is the more general-purpose scaling technique. As a heuristic: use normalization when the algorithm needs bounded inputs; use standardization when the algorithm assumes normally distributed features or when outliers are present.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler

df = pd.DataFrame({
    'Age':    [22, 25, 47, 52, 46, 56, 35, 29],
    'Salary': [25000, 30000, 70000, 80000, 65000, 90000, 55000, 40000],
    'Height': [162, 170, 175, 168, 180, 172, 165, 158]
})

print('Original Data:')
print(df)

minmax = MinMaxScaler()
df_normalized = pd.DataFrame(
    minmax.fit_transform(df),
    columns=[c + '_norm' for c in df.columns]
)
print('\nAfter Min-Max Normalization (range [0,1]):')
print(df_normalized.round(4))

stdscaler = StandardScaler()
df_standardized = pd.DataFrame(
    stdscaler.fit_transform(df),
    columns=[c + '_std' for c in df.columns]
)
print('\nAfter Standardization (mean=0, std=1):')
print(df_standardized.round(4))

print('\nMean after standardization (should be ~0):')
print(df_standardized.mean().round(6))
print('\nStd after standardization (should be ~1):')
print(df_standardized.std().round(4))

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, col in zip(axes, df.columns):
    ax.hist(df[col], bins=8, alpha=0.5, label='Original', color='gray')
    ax.hist(df_normalized[col+'_norm'], bins=8,
            alpha=0.5, label='Normalized', color='steelblue')
    ax.hist(df_standardized[col+'_std'], bins=8,
            alpha=0.5, label='Standardized', color='tomato')
    ax.set_title(col)
    ax.legend(fontsize=7)
plt.suptitle('Original vs Normalized vs Standardized')
plt.tight_layout()
plt.show()
```

```
Original Data:
   Age  Salary  Height
0   22   25000     162
1   25   30000     170
2   47   70000     175
3   52   80000     168
4   46   65000     180
5   56   90000     172
6   35   55000     165
7   29   40000     158

After Min-Max Normalization (range [0,1]):
   Age_norm  Salary_norm  Height_norm
0    0.0000       0.0000       0.1818
1    0.0882       0.0769       0.5455
2    0.7353       0.6923       0.7727
3    0.8824       0.8462       0.4545
4    0.7059       0.6154       1.0000
5    1.0000       1.0000       0.6364
6    0.3824       0.4615       0.3182
7    0.2059       0.2308       0.0000

After Standardization (mean=0, std=1):
   Age_std  Salary_std  Height_std
0  -1.4045     -1.4442     -1.0154
1  -1.1567     -1.2177      0.1880
2   0.6610      0.5947      0.9402
3   1.0741      1.0478     -0.1128
4   0.5783      0.3681      1.6924
5   1.4045      1.5009      0.4889
6  -0.3305     -0.0850     -0.5641
7  -0.8262     -0.7646     -1.6172

Mean after standardization (should be ~0):
Age_std      -0.0
Salary_std    0.0
Height_std    0.0
dtype: float64

Std after standardization (should be ~1):
Age_std       1.069
Salary_std    1.069
Height_std    1.069
dtype: float64
```

Q5 **What is feature engineering? Explain how creating new features can improve model performance.**



ANS:- Feature Engineering is the process of using domain knowledge, creativity, and data understanding to extract, transform, combine, or construct new input variables (features) from existing raw data in order to better represent the underlying problem structure for a machine learning algorithm. It is widely regarded as one of the most impactful activities in applied machine learning. As Andrew Ng observed: "Applied machine learning is basically feature engineering." The fundamental intuition is that models learn patterns from the features they are given. If features do not capture the true underlying signal, even the most sophisticated algorithm will struggle. Conversely, a carefully engineered feature set can allow a simple model to achieve excellent results. Common feature engineering techniques include: Arithmetic / Interaction Features — combining existing features (e.g., BMI = Weight / Height^2); Date-Time Decomposition — extracting Year, Month, Day_of_Week, Is_Weekend from raw timestamps; Binning — grouping continuous values into intervals (Age → Child / Adult / Senior); Aggregation — computing customer-level summaries from row-level transactions; Log Transformations — reducing skewness in income or price distributions; and Text Features — extracting word count, sentiment score, or TF-IDF vectors from text columns. Each technique encodes domain knowledge directly into the feature space, giving the model richer, more informative inputs and consistently improving accuracy, convergence speed, and interpretability.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({
    'CustomerID':    [1, 2, 3, 4, 5],
    'Age':           [23, 45, 34, 28, 60],
    'Weight_kg':     [70, 85, 60, 75, 90],
    'Height_m':      [1.75, 1.80, 1.65, 1.70, 1.78],
```